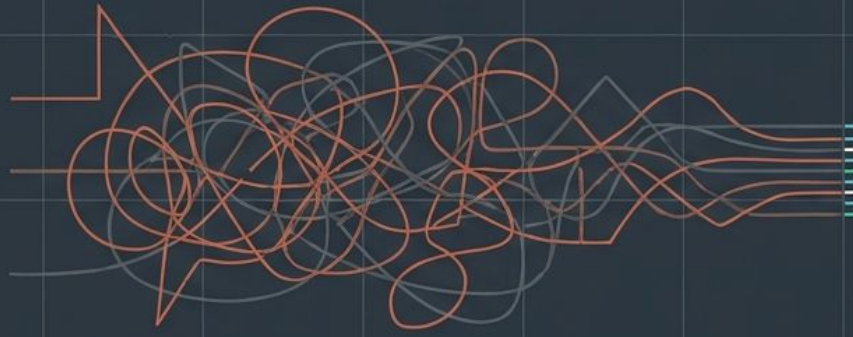
Software Engineering Fundamentals

Understanding the Architecture of Modern Digital Systems



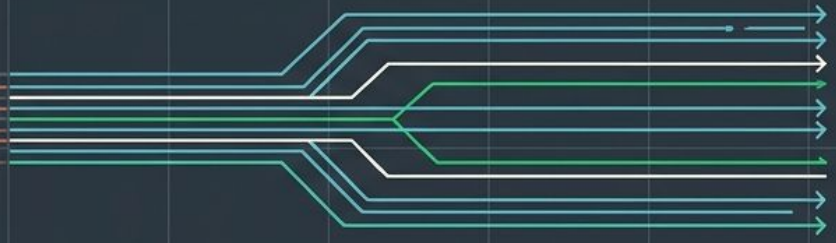
Moving from unpredictable chaos to structured scale

Ad-Hoc Development



- Poor code quality
- Frequent failures
- Difficult maintenance
- Delayed releases
- High development costs

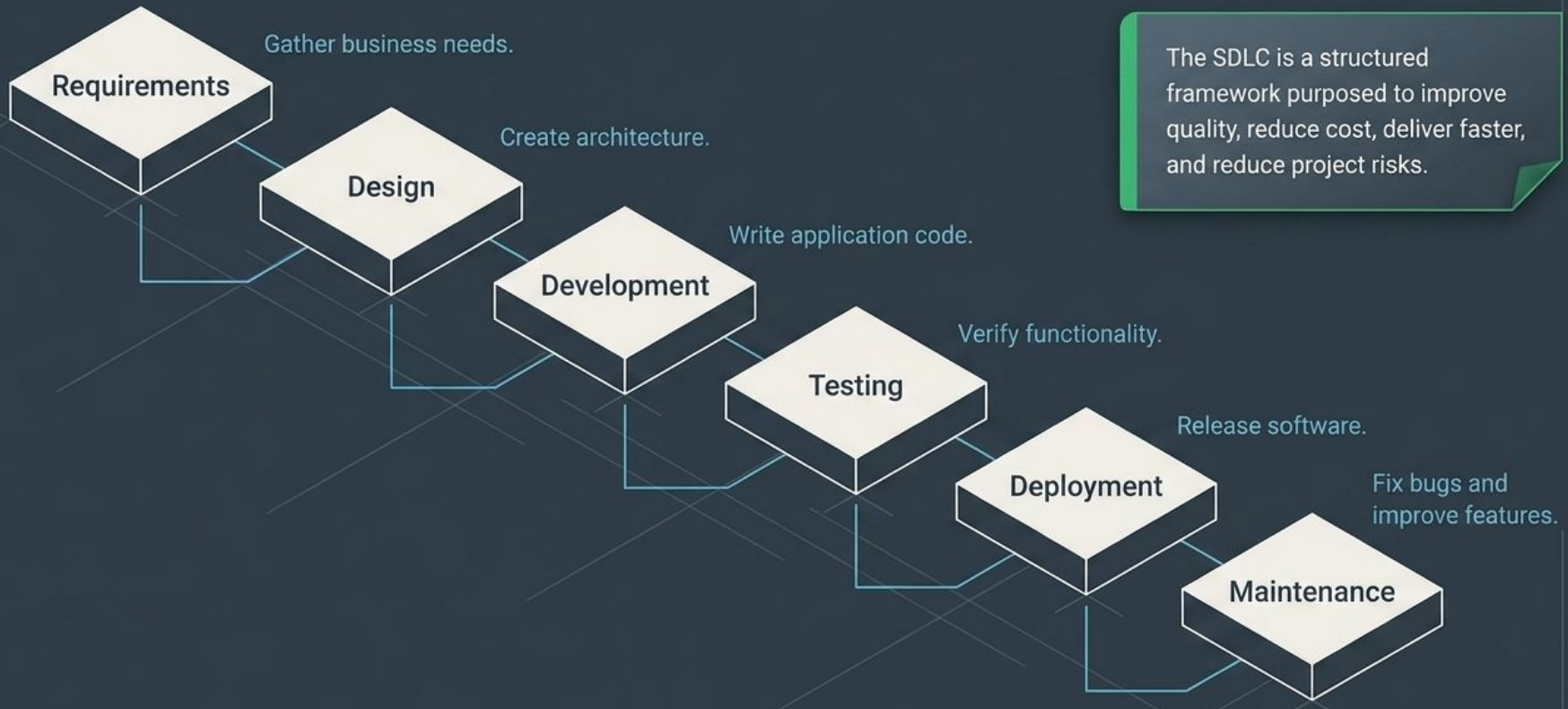
Disciplined Engineering



- Predictable delivery
- Structured development
- Better collaboration
- Higher quality software
- Easier maintenance

Software Engineering is the disciplined process of **designing, building, testing, deploying, and maintaining** software systems through a combination of programming, automation, and continuous improvement.

The blueprint for structured development



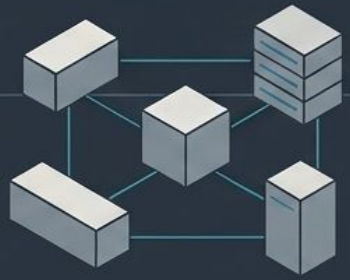
The SDLC is a structured framework purposed to improve quality, reduce cost, deliver faster, and reduce project risks.

The evolution driven by the demand for scale

The Breaking Point

Traditional development resulted in communication gaps, configuration mismatches, slow bug fixes, and high downtime because Development and Operations worked entirely separately.







1970–1990

Standalone desktop software, small development teams, and manual deployments.

1990–2010

The shift to Web applications, client-server architectures, and larger databases.

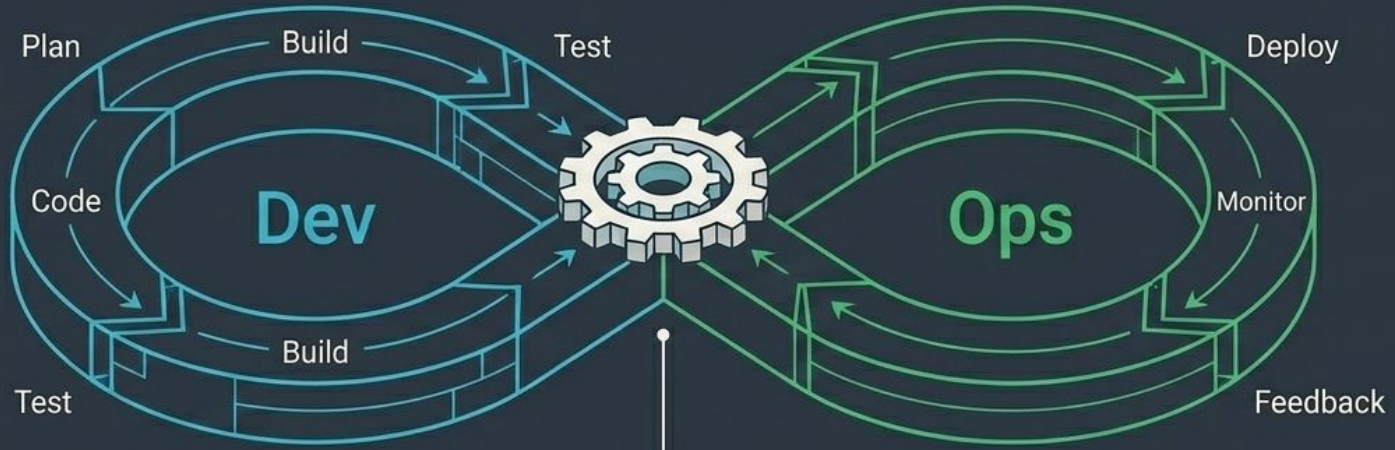
2010–Present

The era of Cloud Computing, DevOps, Containers, Kubernetes, Microservices, and AI-assisted development.

Traditional Model: Developer → QA → Operations

(Plagued by manual deployments and slow release cycles).





The DevOps Advantage

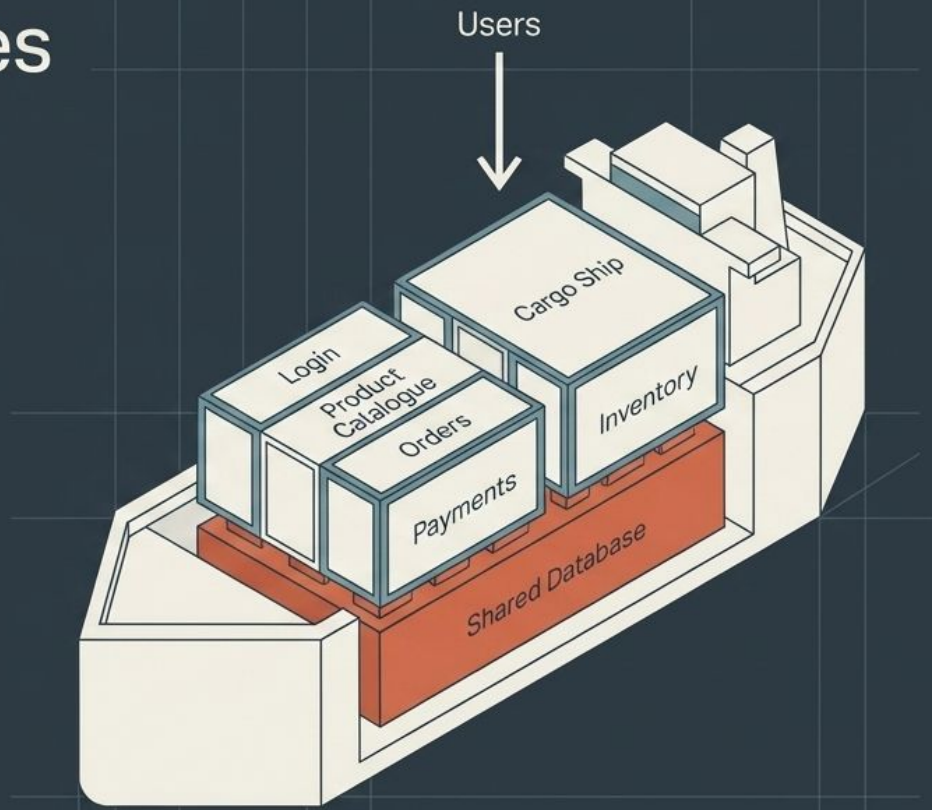
Integrates Developer ↔ QA ↔ Operations ↔ Automation.
Results in faster releases and highly collaborative environments.

The Monolith packages everything into a single codebase

A monolithic architecture contains all business functionalities inside a single codebase, sharing a single database.

The Metaphor:

Like a massive cargo ship, it carries everything and is simple to launch initially, but it turns slowly. If the engine fails, the entire ship stops.

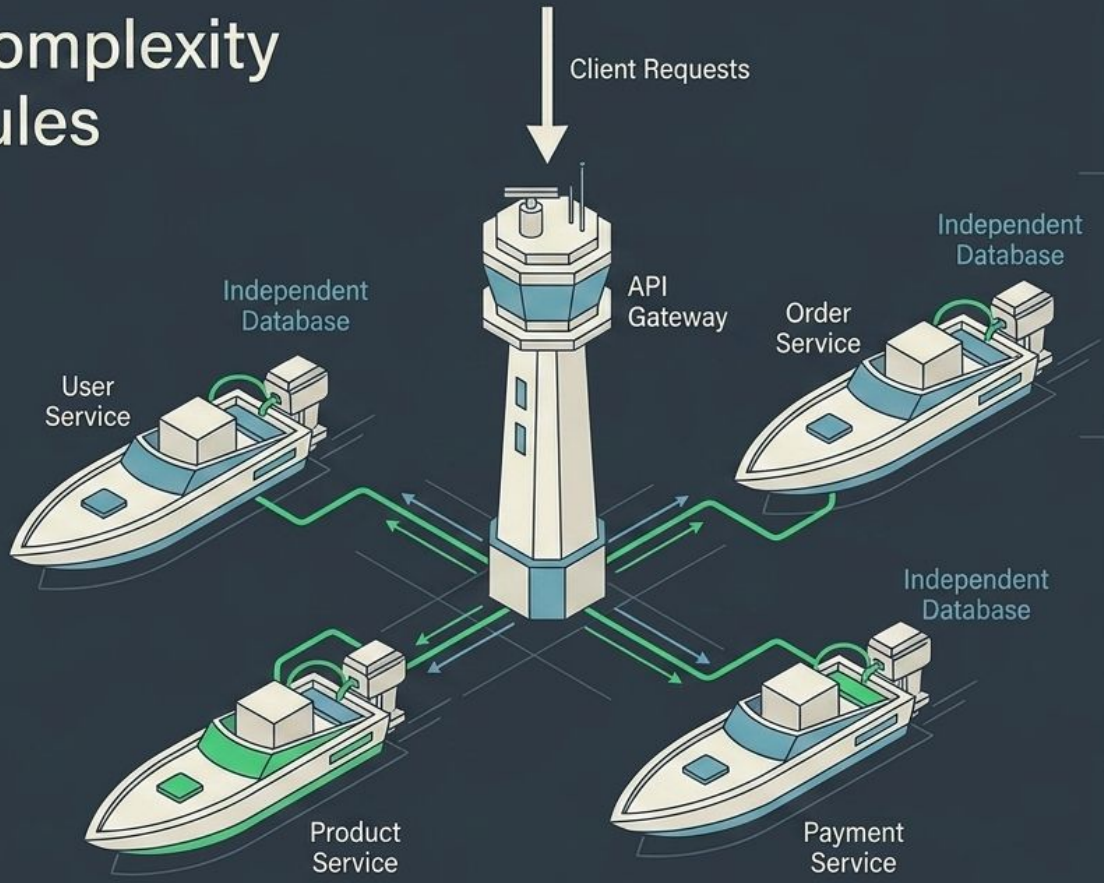


Microservices divide complexity into independent modules

Microservices split an application into multiple independent services that communicate using APIs.

The Metaphor:

Like a fleet of agile speedboats, they are fast and independent. If one boat breaks down, the rest of the fleet keeps moving.



Architecture diagnostic matrix

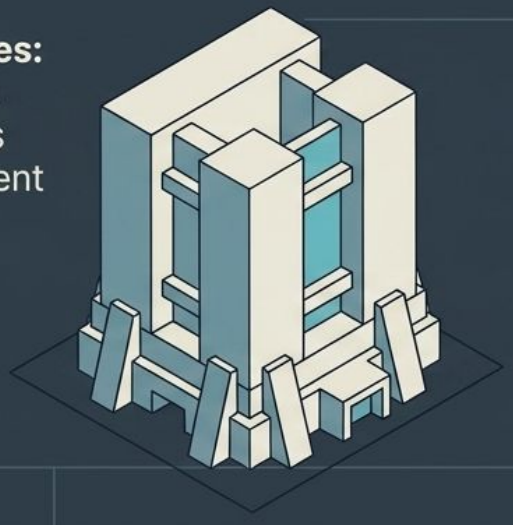
	Monolithic	Microservices
Deployment	Deploy the entire app	Independent deployment
Scaling	Scales the entire app	Scales individual services
Database	Shared database	Separate, independent databases
Fault Isolation	Low (single point of failure)	High fault isolation
Maintenance & CI/CD	Harder as teams grow	Easier CI/CD, smaller codebases
Complexity	Simple initially	Complex network communication, service discovery challenges

Matching structural blueprints to business realities

When to build a Monolith

Suitable for startups, internal business tools, and environments with lower operational complexity.

Real-World Examples:
- Internal HR Portals
- Small ERP systems
- College Management Systems



When to build Microservices

Essential for massive scalability, distributed systems, and large engineering teams.

Real-World Examples:
- Netflix
- Amazon
- Uber
- Spotify
- Food Delivery Platforms



The layers of the modern enterprise technology stack



Frontend (User Interface)
Built with tools like React.

Backend API
The logic layer, routing via REST to frameworks like Spring Boot.

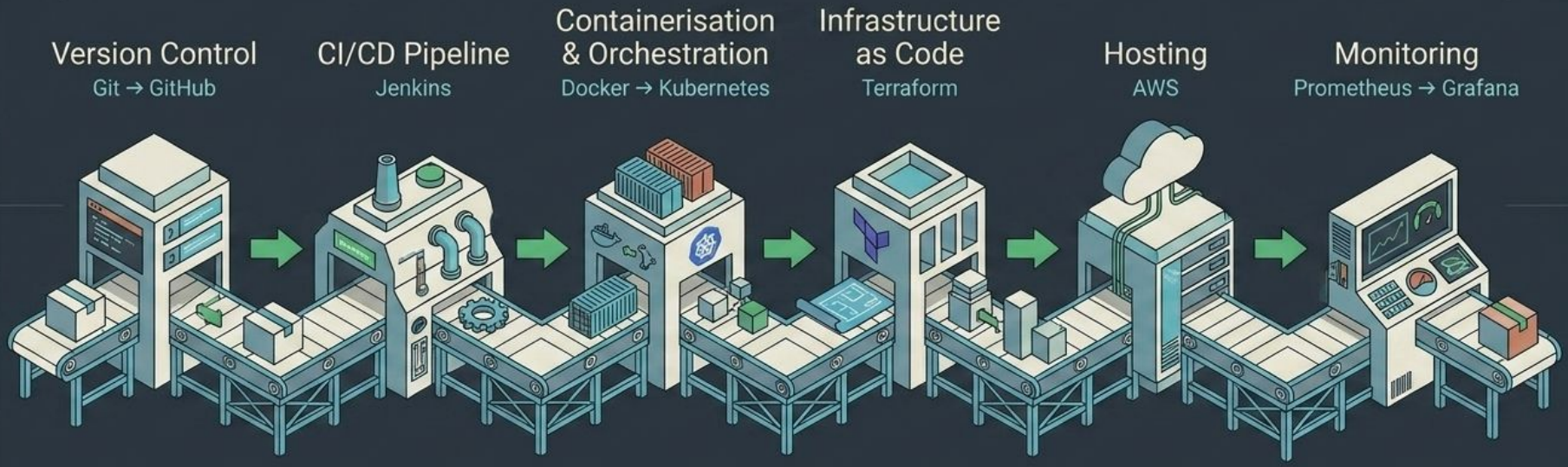
Data & State
Storing and recalling information quickly using PostgreSQL (Database) and Redis (Cache).

Messaging
Managing asynchronous communication via Kafka.

Infrastructure
Packaging and running the application using Docker (Containers), Kubernetes (Orchestration), and AWS (Cloud).

Observability
Monitoring the system health with Prometheus and Grafana.

Automating the process with the DevOps toolchain



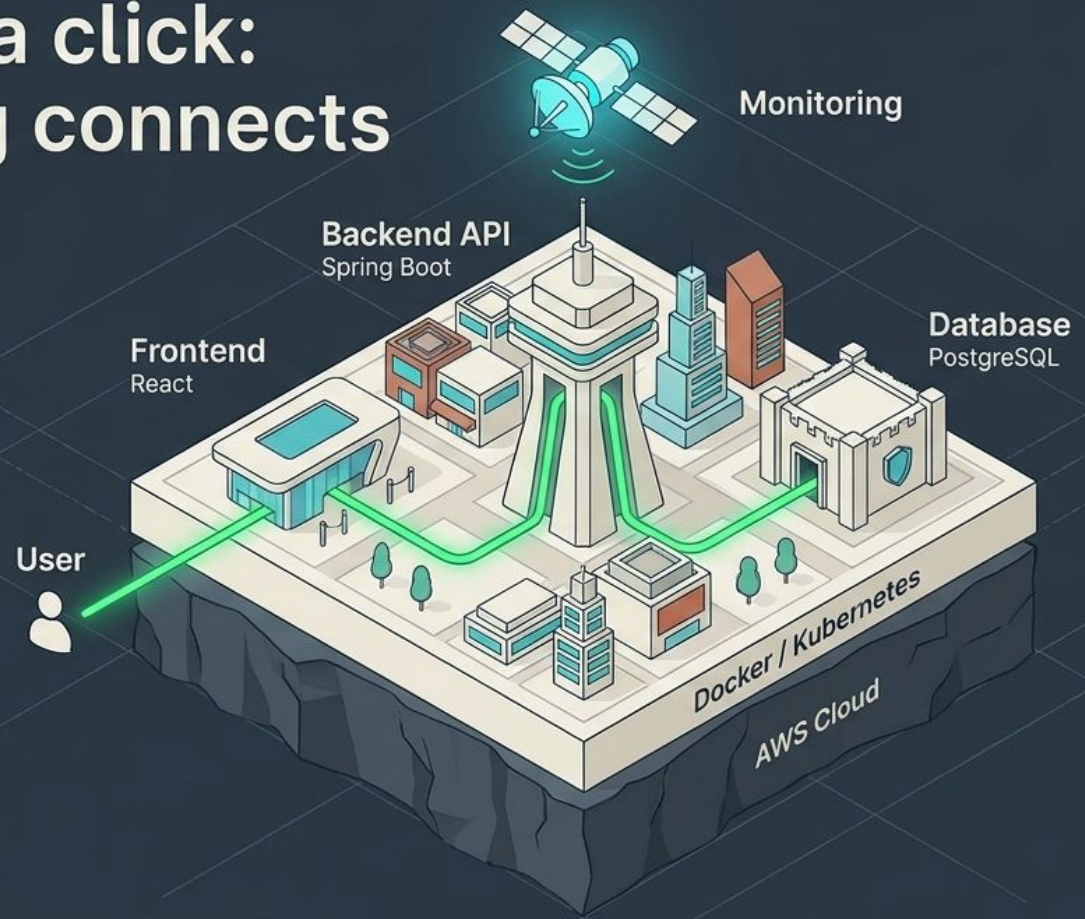
The journey of a click: How everything connects

Process: Software Engineering provides the discipline and the SDLC framework.

Engine: DevOps automates the delivery and continuous integration.

Structure: Microservices divide the architecture into scalable components.

Infrastructure: The Cloud provides the raw computing power to run it all.



Foundations established for modern scale

Key Takeaways

- ✓ Software Engineering shifts development from chaos to predictable, structured delivery.
- ✓ Monolithic architecture is simple to start, but Microservices are required for independent scaling and fault tolerance.
- ✓ The SDLC defines the lifecycle, while DevOps transforms it into a continuous loop for faster releases.
- ✓ Modern applications rely on a multi-layered technology stack powered by cloud infrastructure and automated toolchains.

Fluency Check (Can you answer these?)

1. How does the SDLC differ from DevOps?
2. Why would a startup choose a Monolith over Microservices?
3. What role does an API Gateway play in Microservices?
4. Where do Docker and Kubernetes fit into the delivery pipeline?

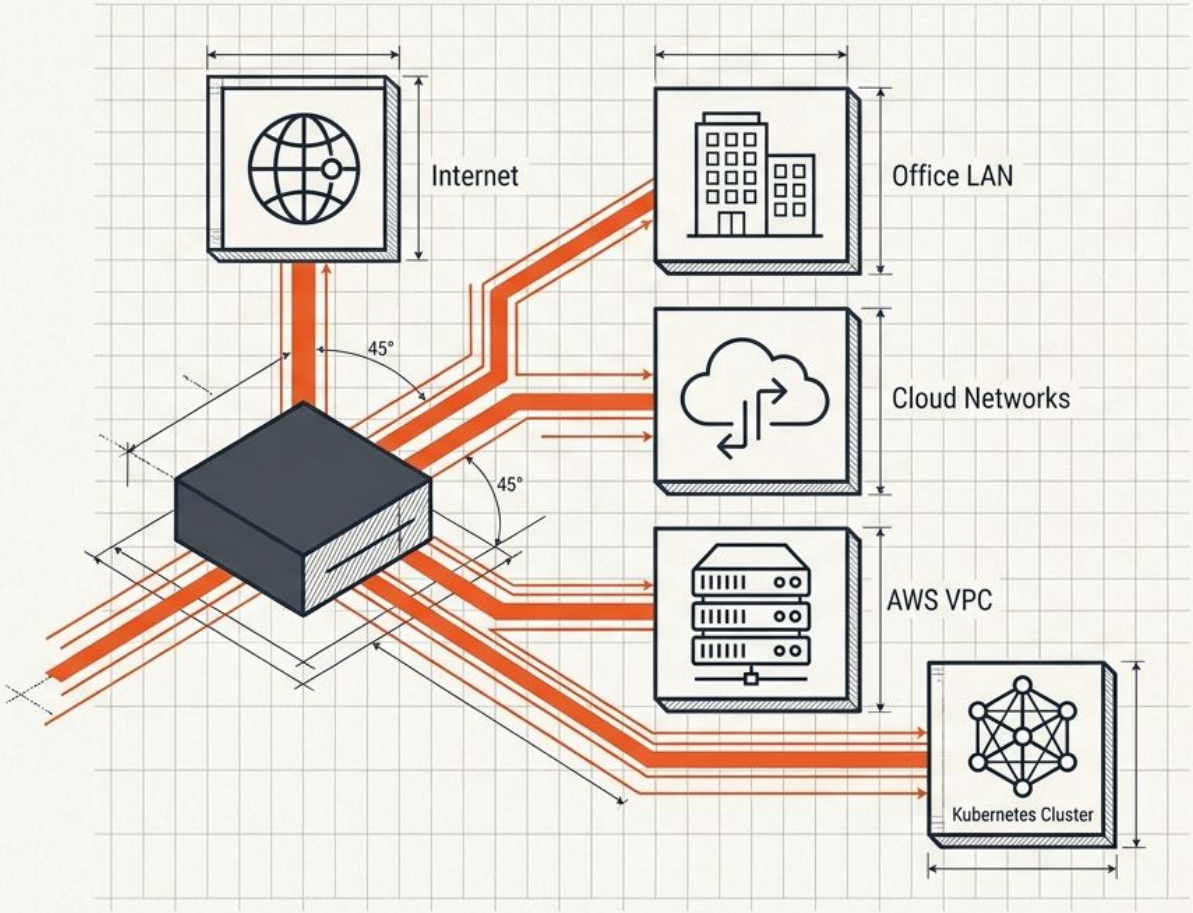
Networking Fundamentals

Module 2

- OSI Model
- TCP/IP
- DNS
- HTTP & HTTPS
- Ports
- Load Balancer
- Reverse Proxy
- CDN

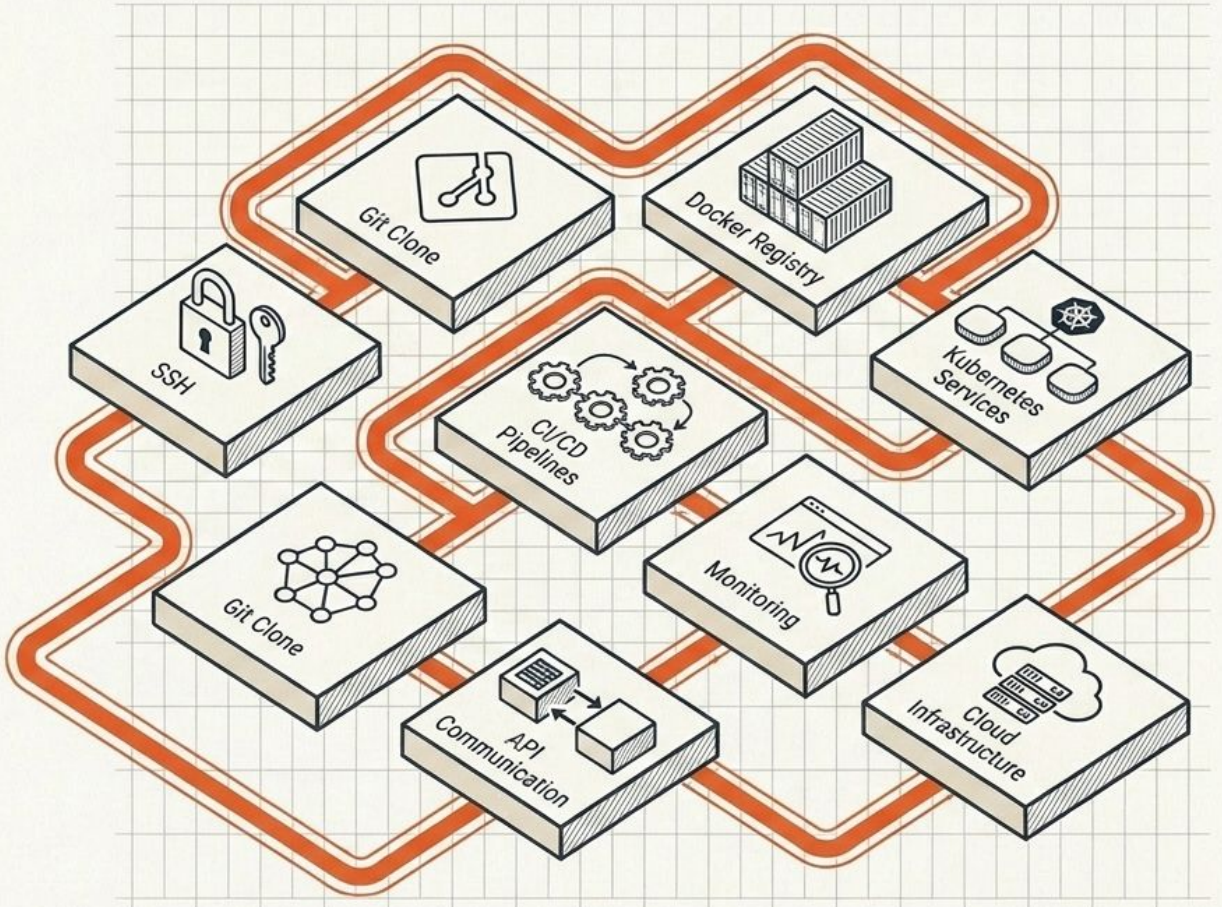
Defining Computer Networking

Networking allows multiple computers and devices to communicate and share resources.

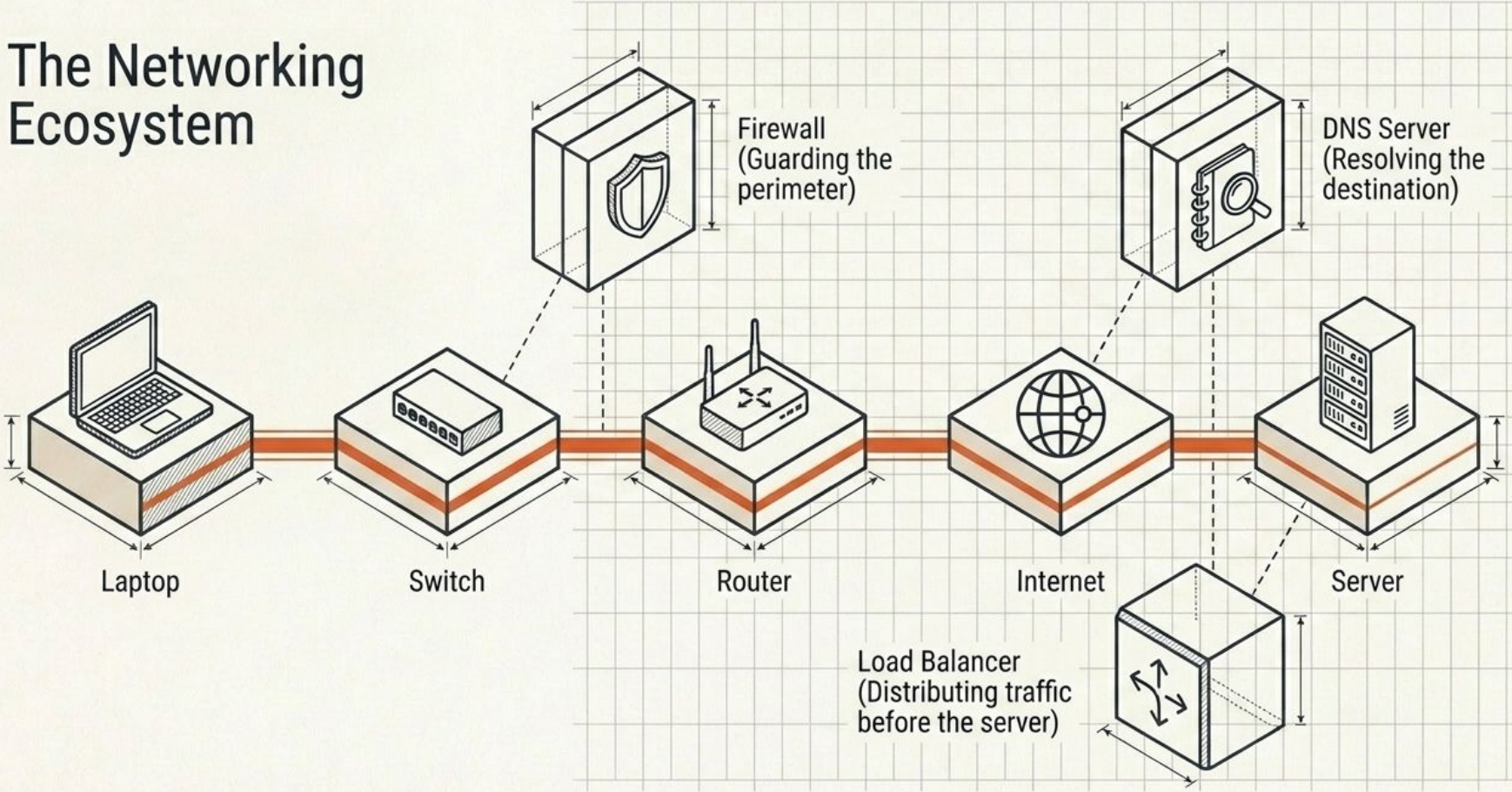


The Backbone of Modern DevOps

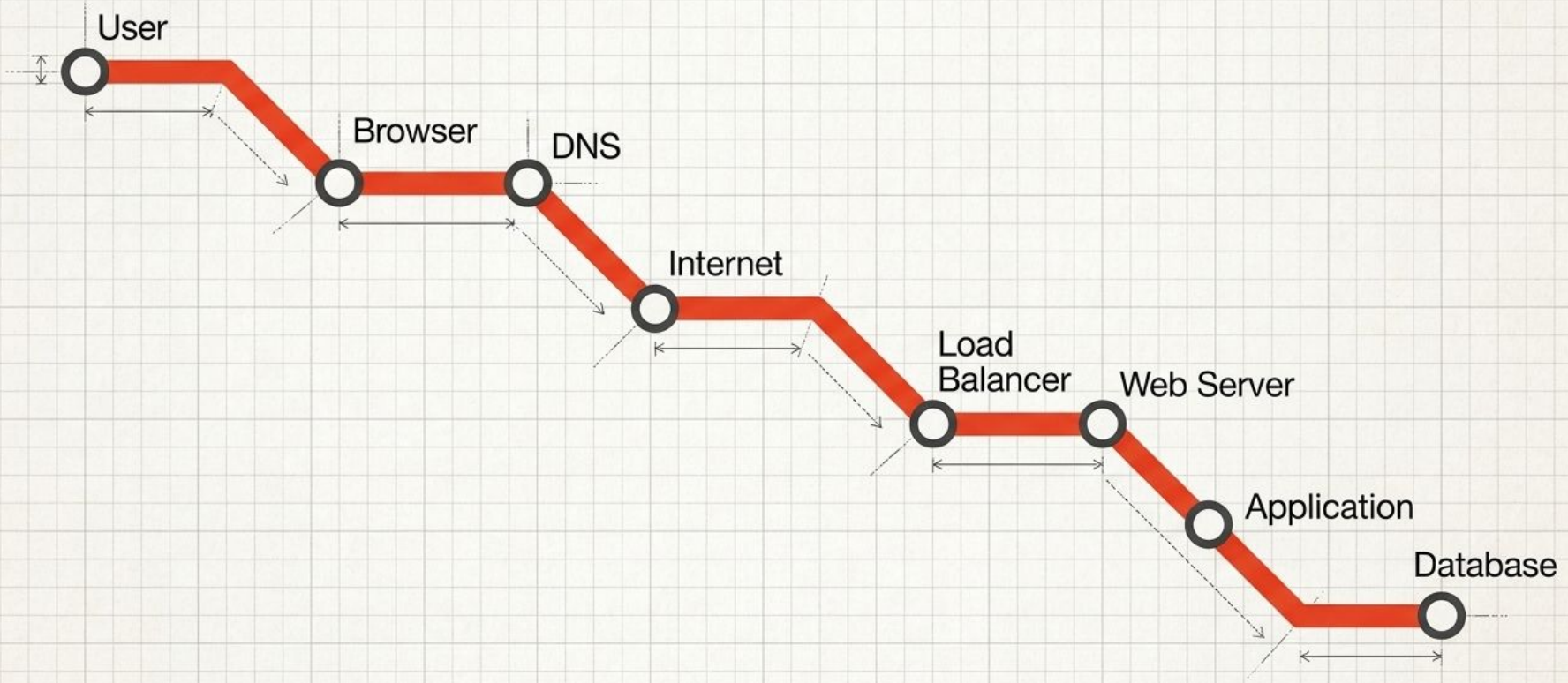
Networking is actively involved in every stage of the lifecycle.



The Networking Ecosystem



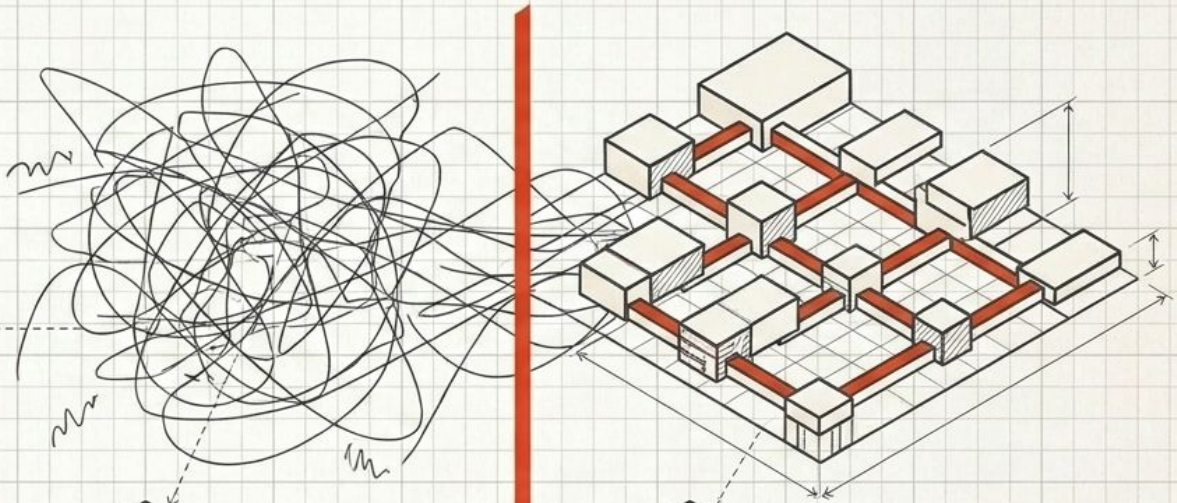
How Data Travels

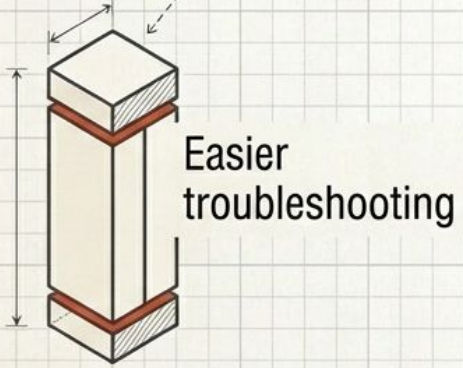


Project Code 16:9

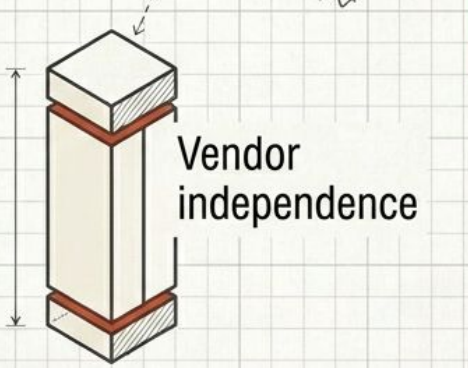
The Need for a Standardised Blueprint

The OSI Model standardises communication between diverse systems.

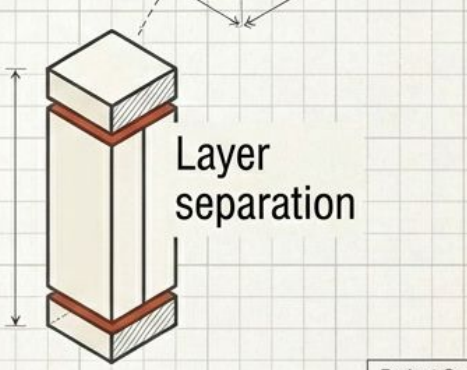




Easier troubleshooting



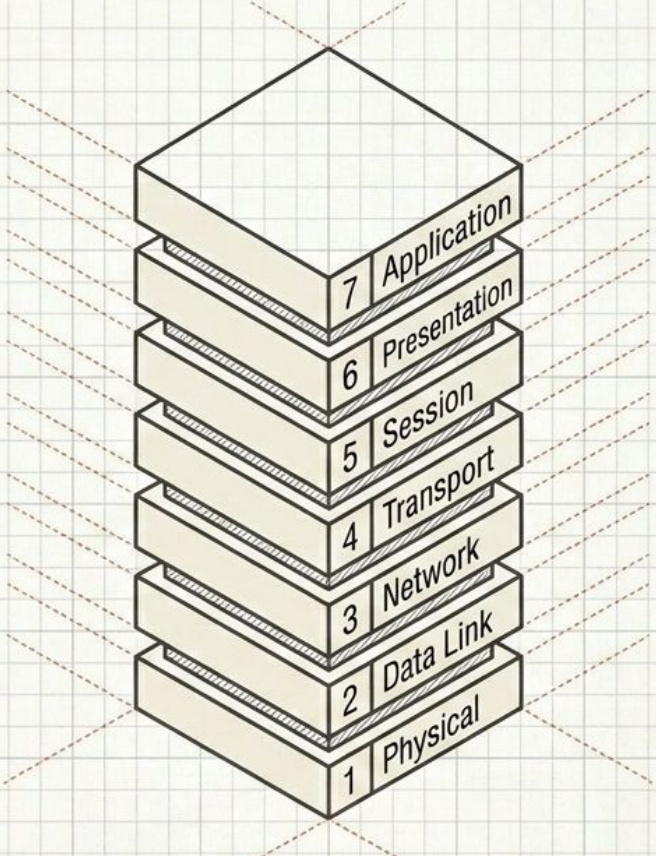
Vendor independence

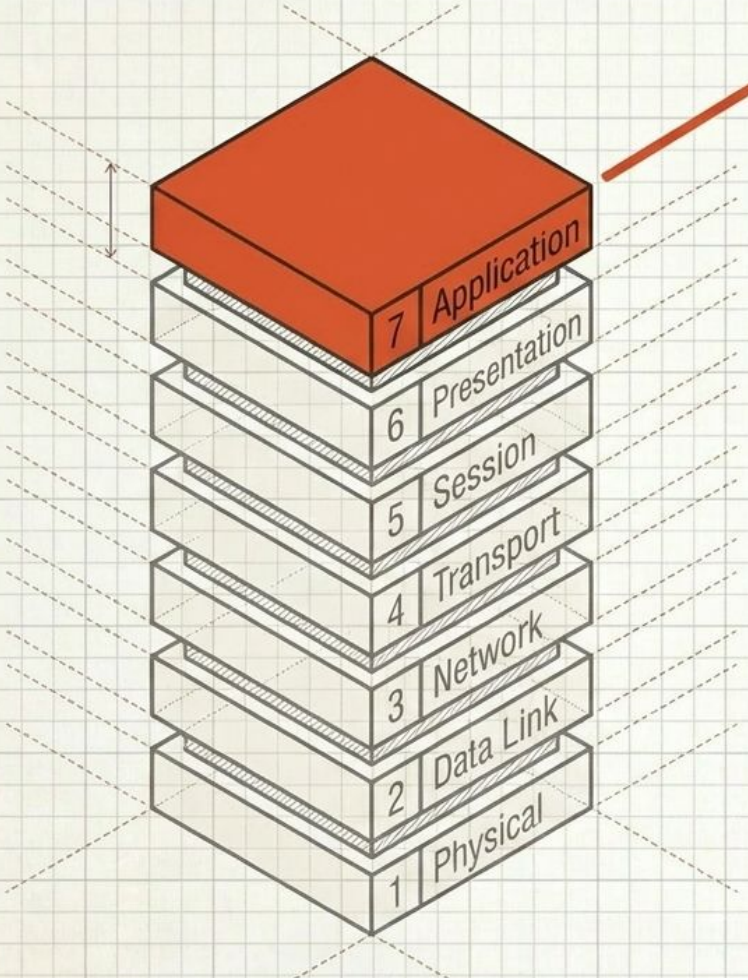


Layer separation

Project Code 16:9

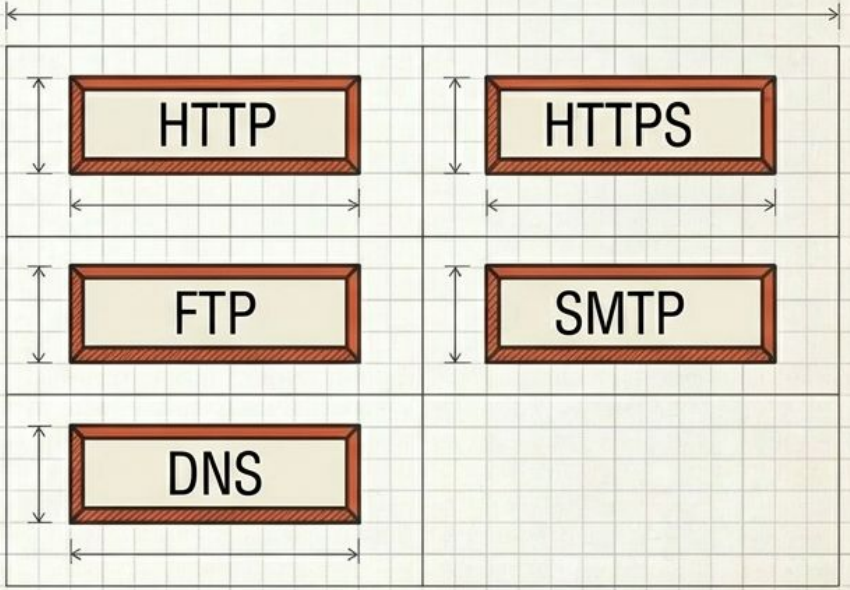
The 7 Layers of the OSI Model

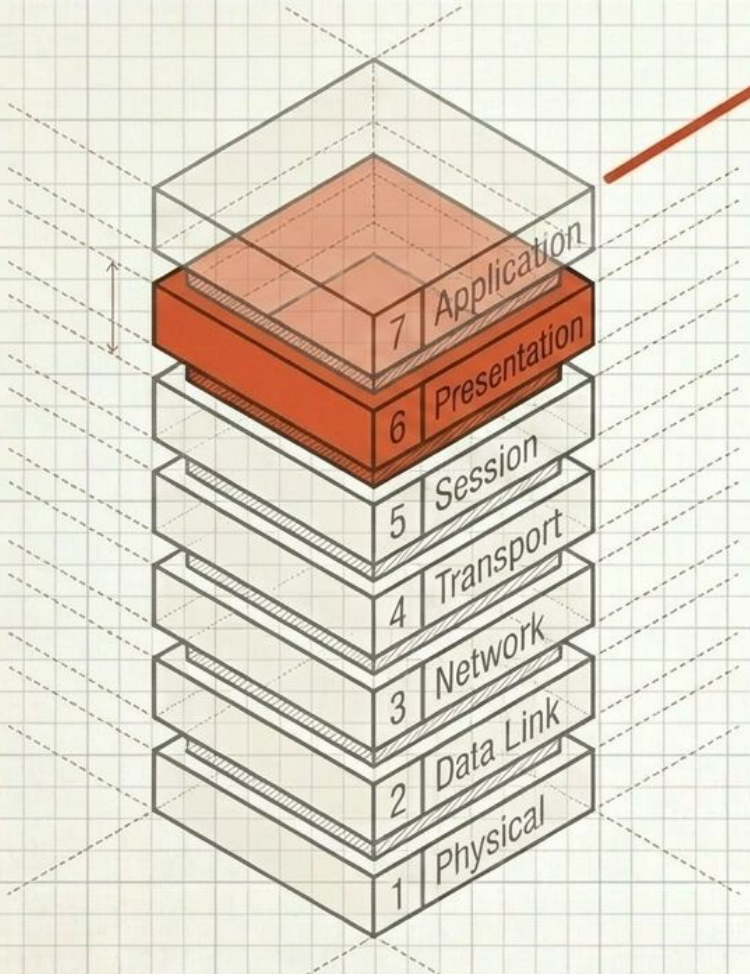




Layer 7 – Application

This is where the user directly interacts with the network.

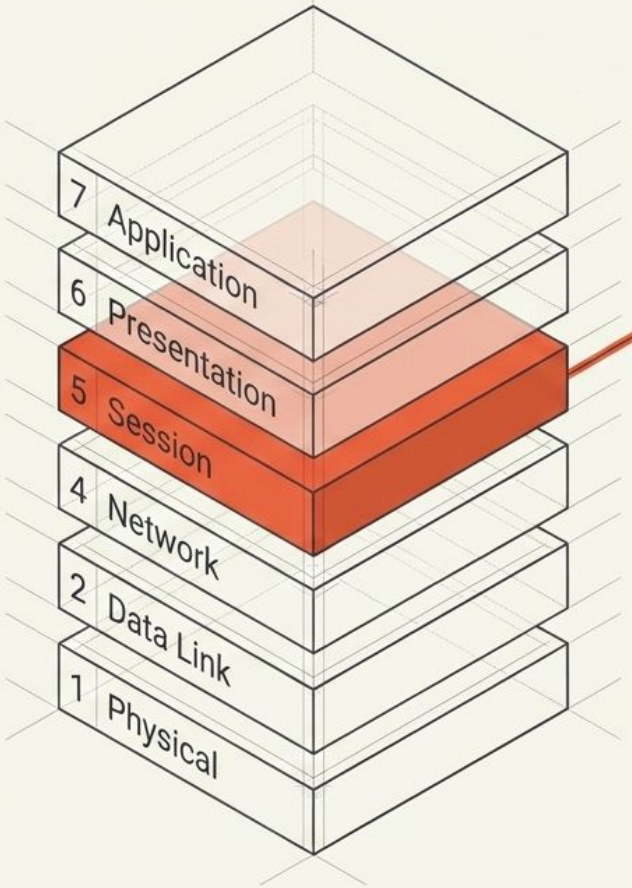




Layer 6 – Presentation

	Encryption
	Compression
	Data Formatting

Example: SSL/TLS



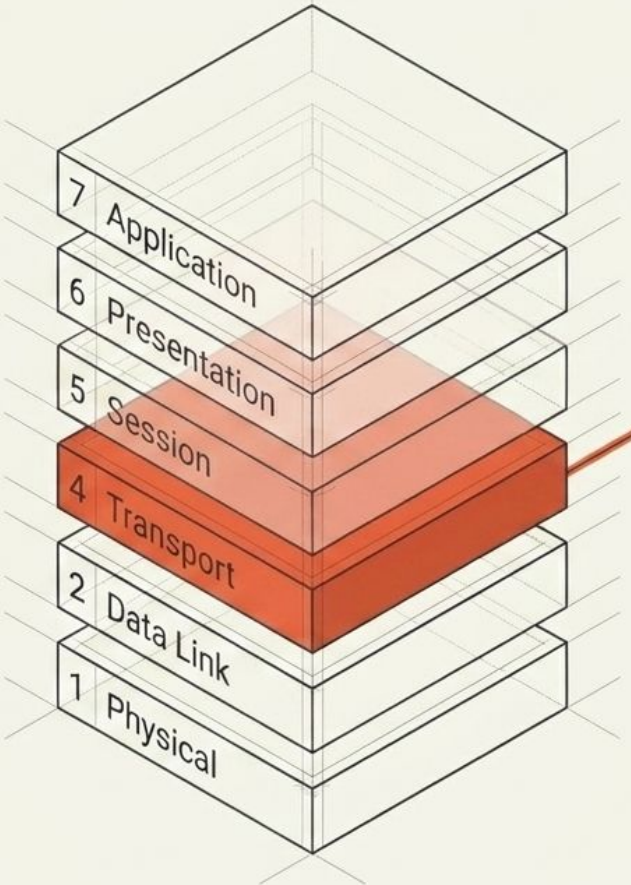
Layer 5 – Session

1 Session creation

2 Session management

3 Session termination

Project Code 16:9

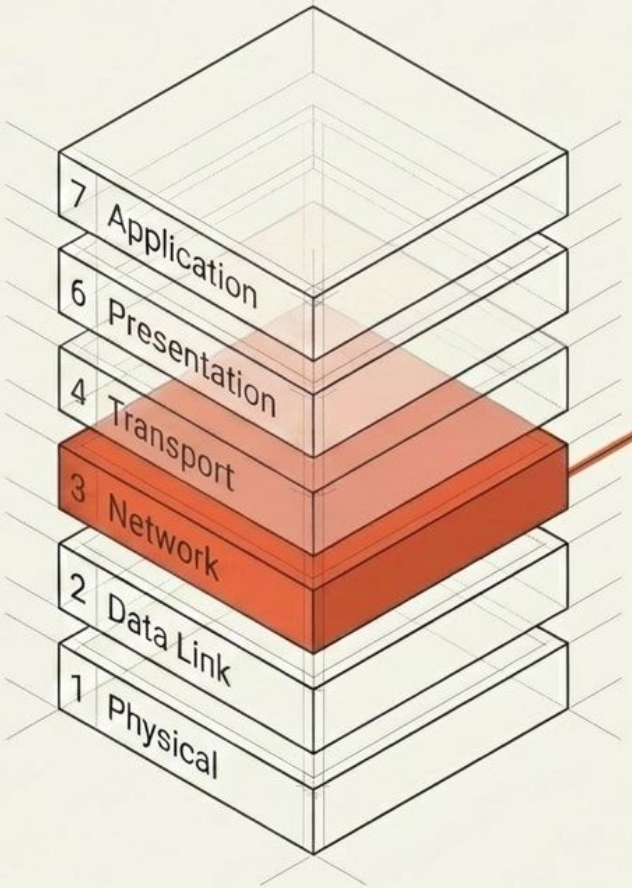


Layer 4 – Transport

- Reliable communication
- Port numbers

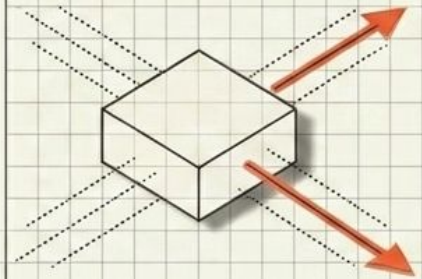
TCP

UDP



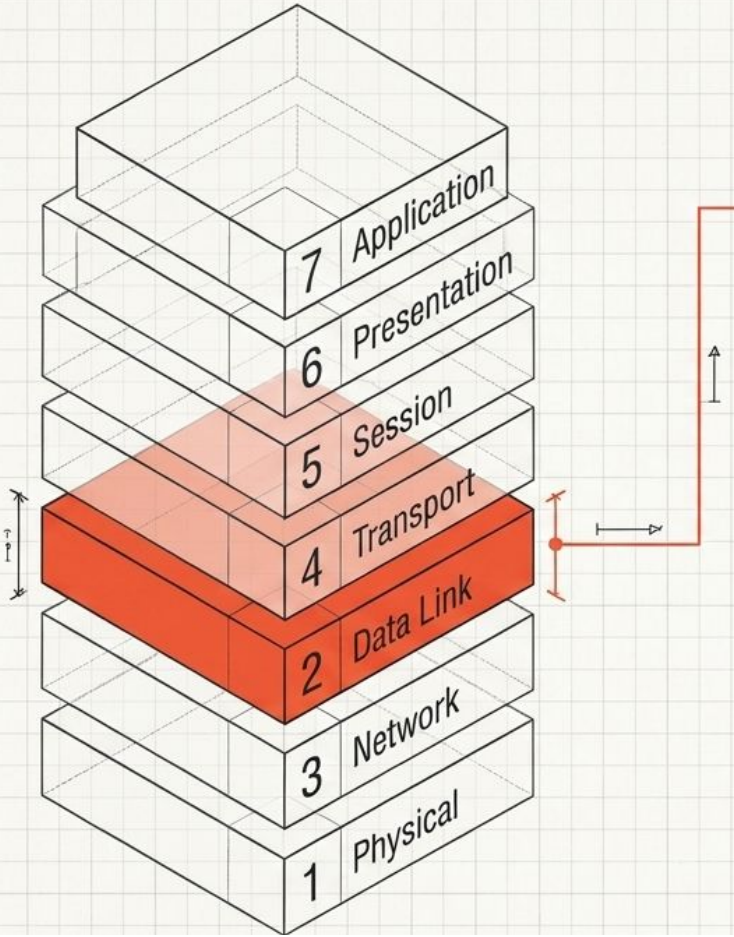
Layer 3 – Network

- Responsible for routing data across the ecosystem.



IP Address

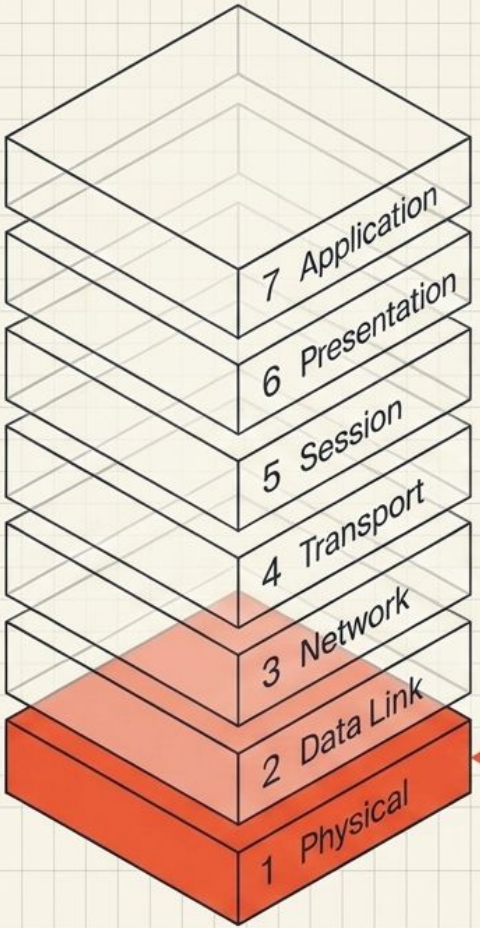
Router



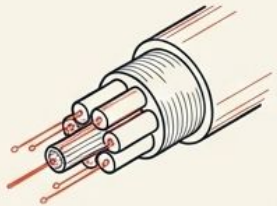
Layer 2 – Data Link



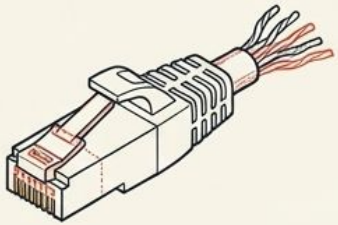
- MAC Address addressing
- Frame delivery
- Switch communication



Layer 1 – Physical



Fibre



Ethernet Cable



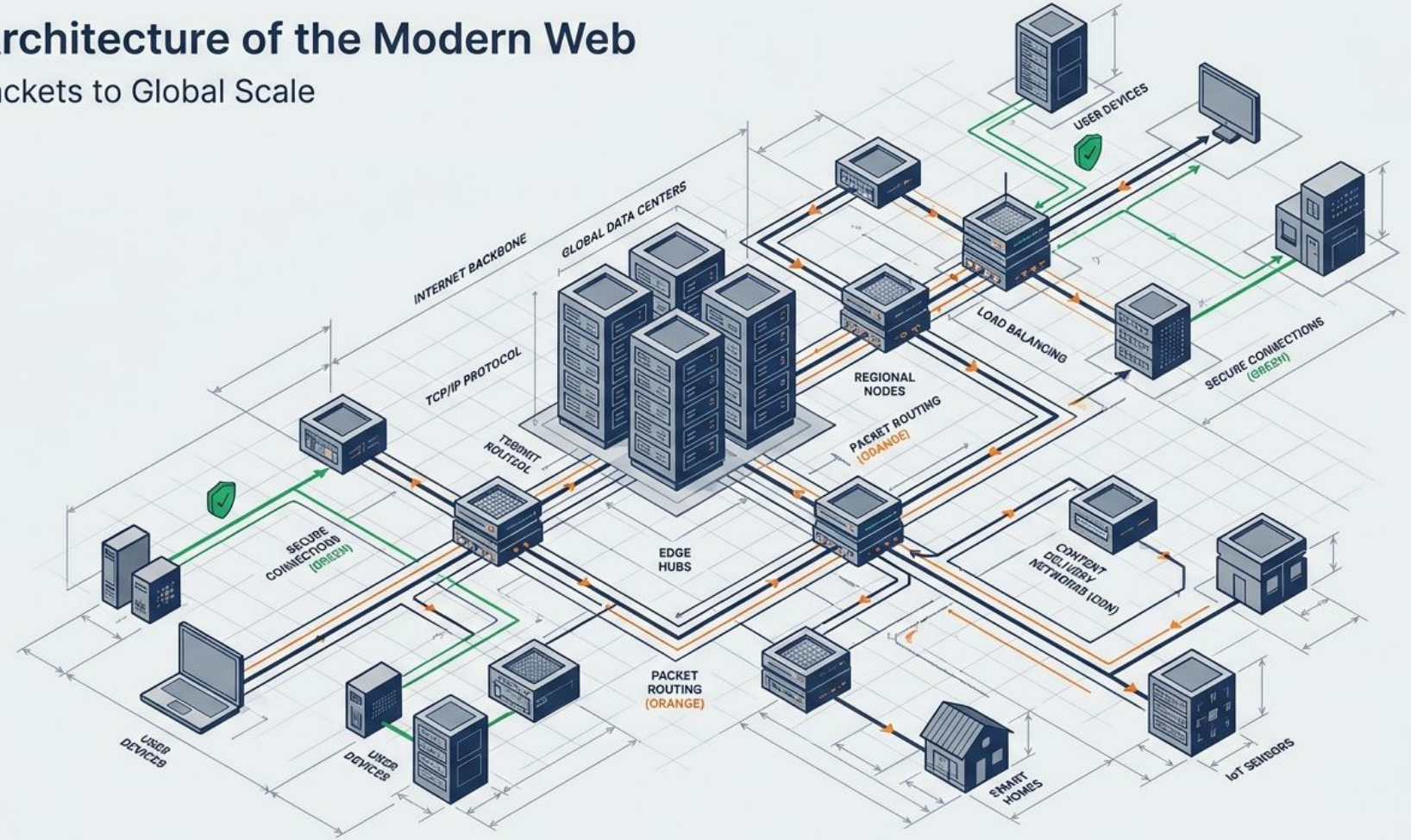
WiFi Signals

The Master OSI Matrix

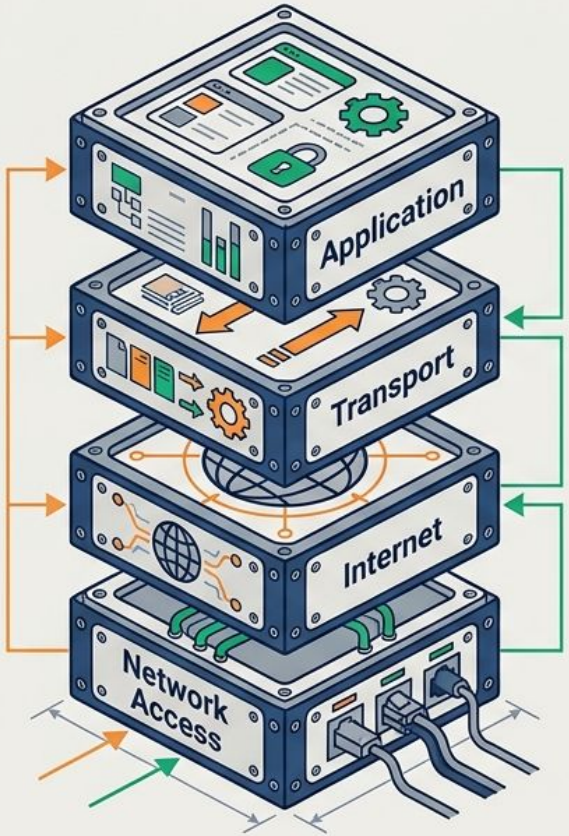
Application	HTTP, HTTPS, DNS, FTP
Presentation	SSL, TLS
Session	NetBIOS
Transport	TCP, UDP
Network	IP, ICMP
Data Link	Ethernet
Physical	Cable

The Architecture of the Modern Web

From Packets to Global Scale



Establishing the Rules of Internet Communication



Why TCP/IP? It is the practical implementation of networking and the standard used by the Internet.

OSI	TCP/IP
7 Layers	4 Layers
Conceptual	Practical
Educational	Internet Standard

Choosing How Data Travels at the Transport Layer

TCP (Transmission Control Protocol)



- Characteristics:
  - Reliable, Connection-Oriented (Three-way handshake), Ordered, Slow (due to error checking).
- Use Cases:
  - SSH, HTTPS, Databases.

UDP (User Datagram Protocol)



- Characteristics:
  - Fast, Connectionless, Unordered, Faster (No acknowledgment, no retransmission).
- Use Cases:
  - Video Streaming, Gaming, VoIP, DNS Queries.

Translating Human Intent into Machine Addresses

google.com → [DNS Translation] → **142.x.x.x**

Common DNS Record Types

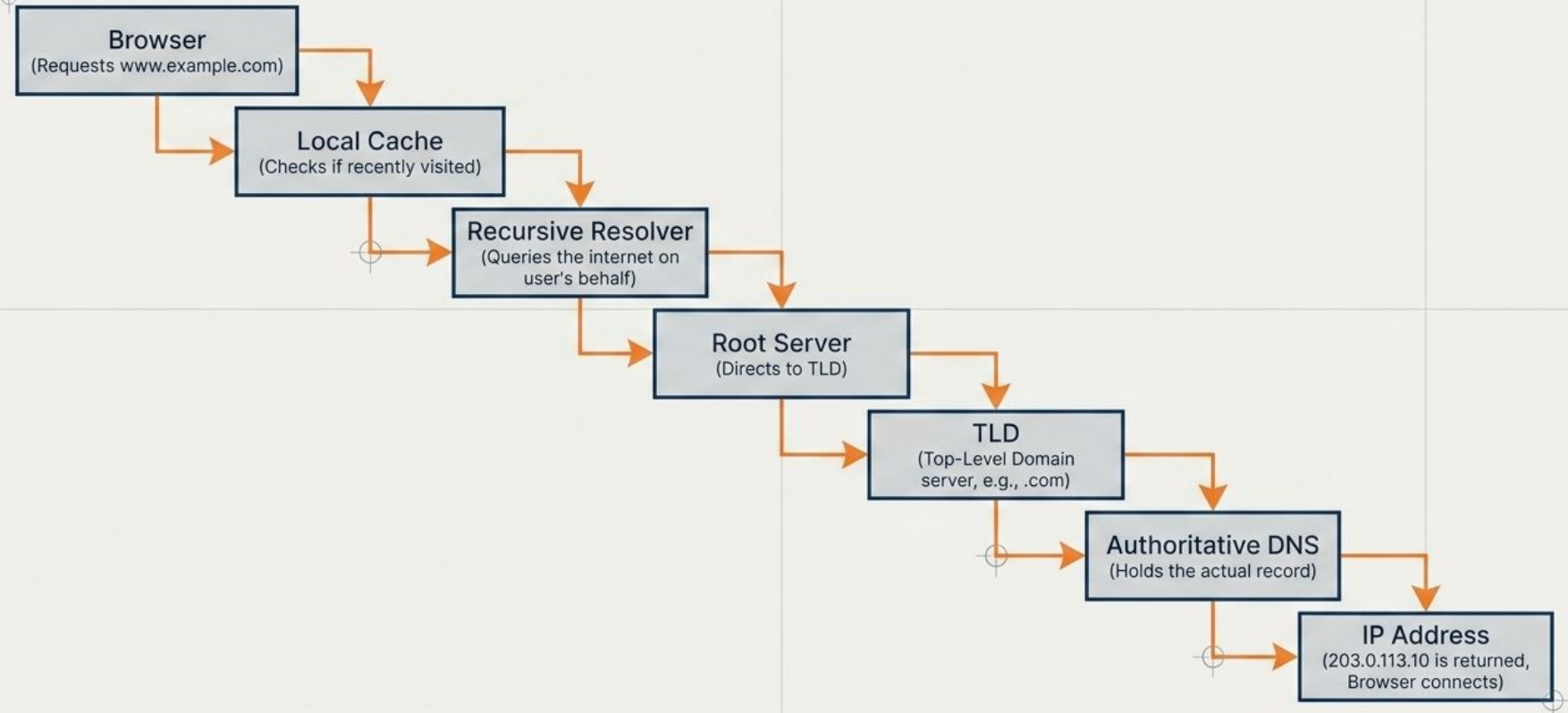


Modern Cloud Implementation (AWS Route53)

- Private Hosted Zones
- Public Hosted Zones
- Health Checks

Common Interview Focus: Know the precise differences between A Record, CNAME, MX, and TXT.

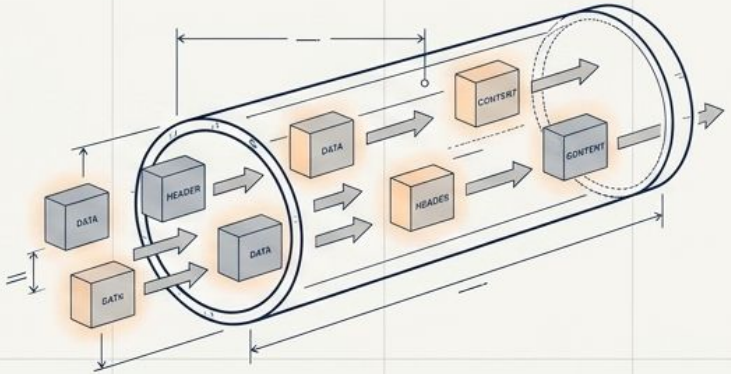
The DNS Resolution Waterfall



Securing the Payload Across the Network

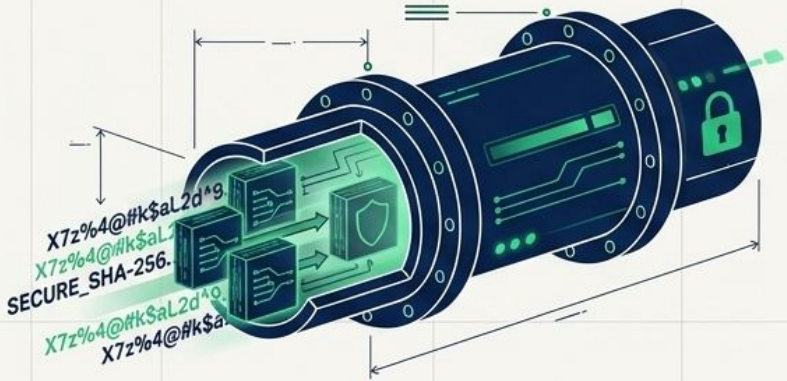
HTTP (HyperText Transfer Protocol)

- Port 80 | Plain Text
- No Encryption
- Less Secure

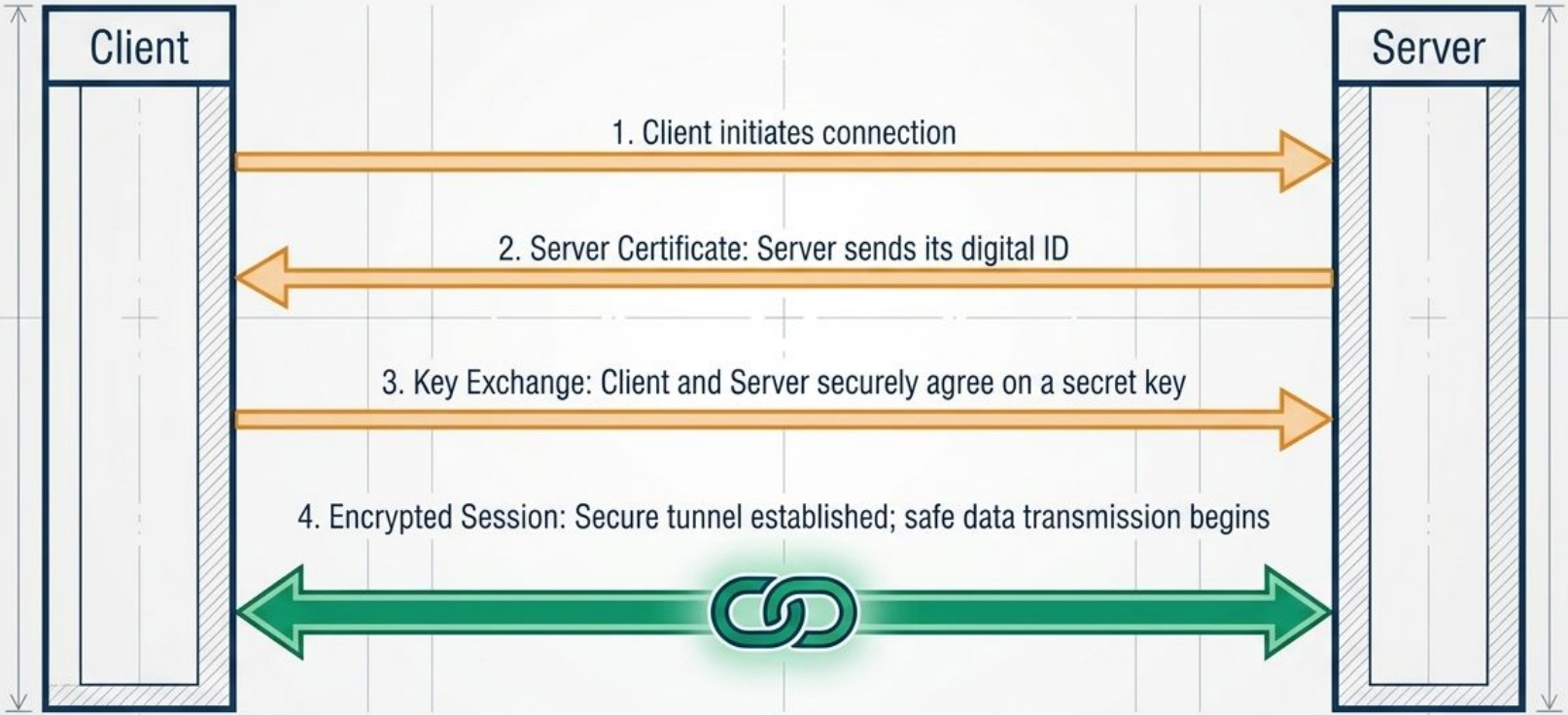


HTTPS (HyperText Transfer Protocol Secure)

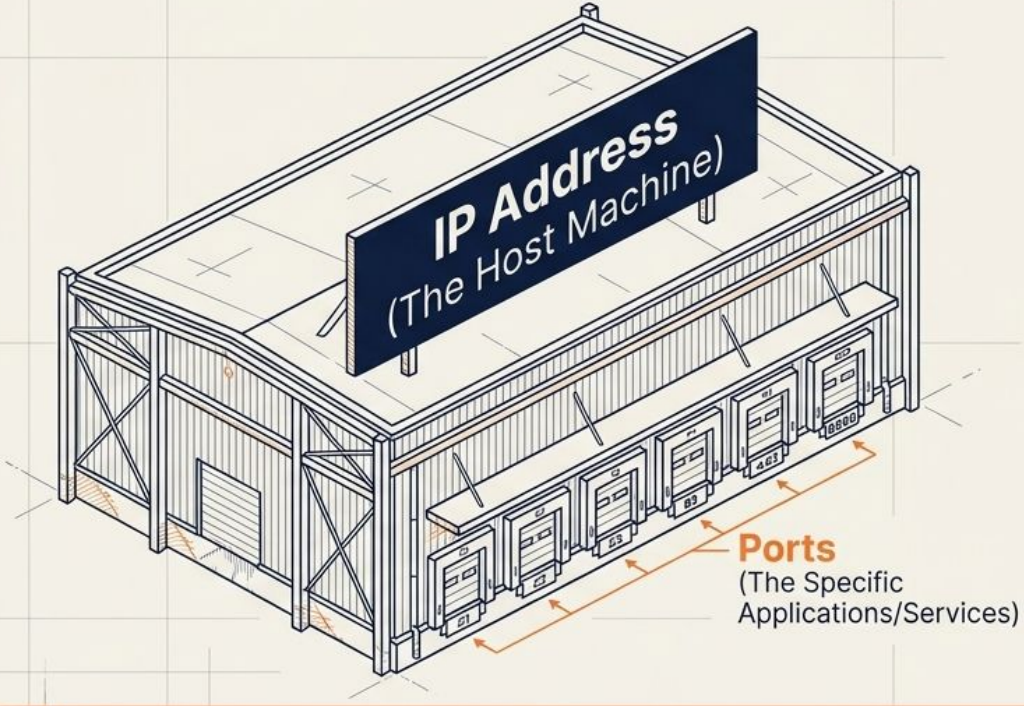
- Port 443
- SSL/TLS Encryption
- Secure Communication



Establishing Trust Before Transmission



Targeting the Specific Application Endpoint



The Distinction:
An IP identifies the host machine. A port identifies the specific service running on that machine.

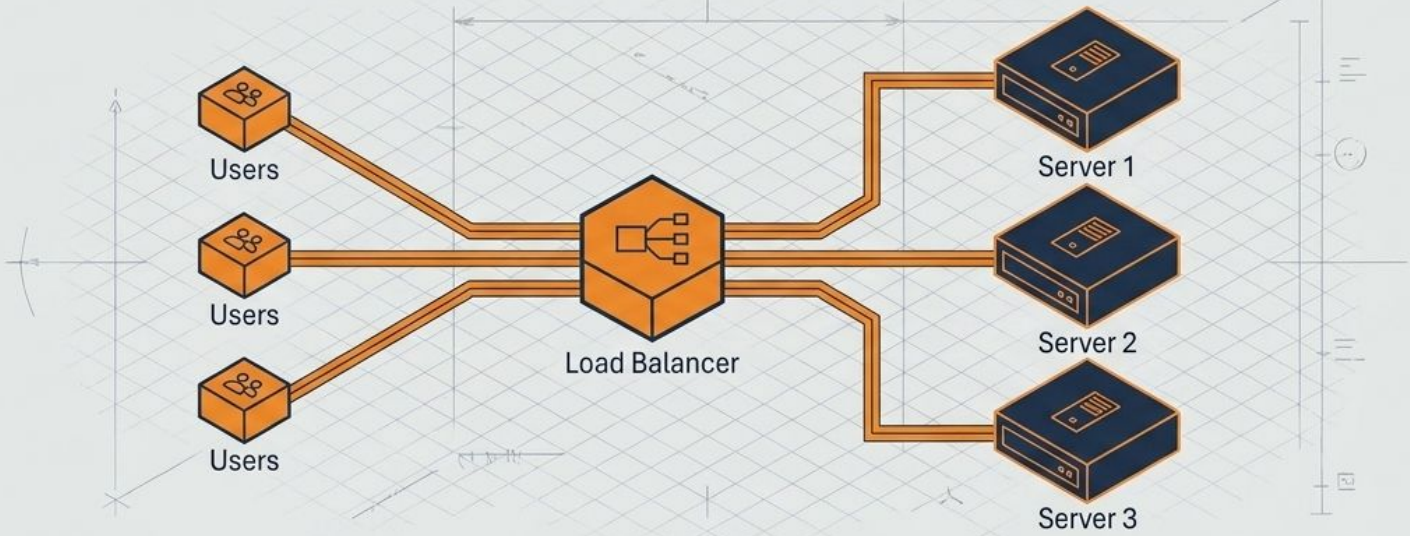
Interview Focus
Know the difference between Port, IP, and Socket.

Troubleshooting Toolkit: `netstat -tulnp` | `ss -tulnp` | `lsof -i` | `telnet` | `nc` | `curl`

The Essential Port Directory

Standard Services		DevOps Ecosystem
21 – FTP	110 – POP3	3000 – Grafana
22 – SSH	143 – IMAP	5000 – Docker Registry
25 – SMTP	443 – HTTPS	6443 – Kubernetes API
53 – DNS	3306 – MySQL	8080 – Jenkins
80 – HTTP	5432 – PostgreSQL	9000 – SonarQube
110 – POP3	6379 – Redis	9090 – Prometheus
143 – IMAP	27017 – MongoDB	

Distributing the Load for High Availability



Core Concept

Distributes incoming traffic across multiple servers.

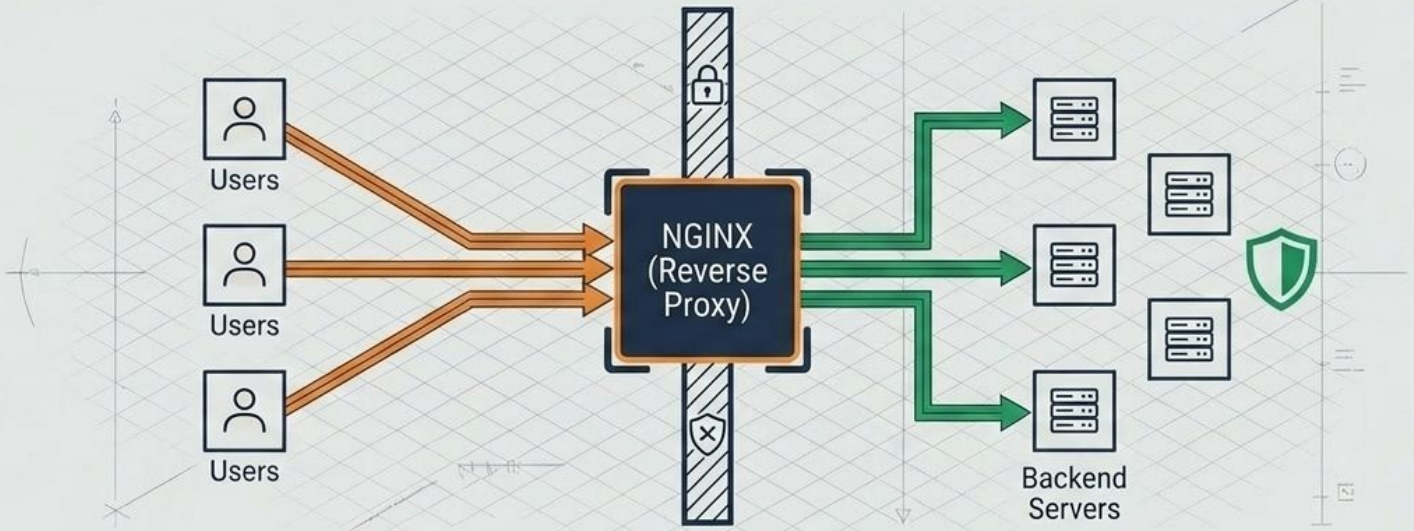
Key Benefits

High Availability, Scalability, Fault Tolerance.

Routing Algorithms

Round Robin, Least Connections, Weighted, IP Hash.

Shielding the Backend Architecture



Core Concept

A Reverse Proxy acts on behalf of backend servers, preventing direct client access to the infrastructure.

Capabilities & Benefits

- SSL Termination
- Security & Anonymity
- Caching
- Load Balancing
- URL Routing

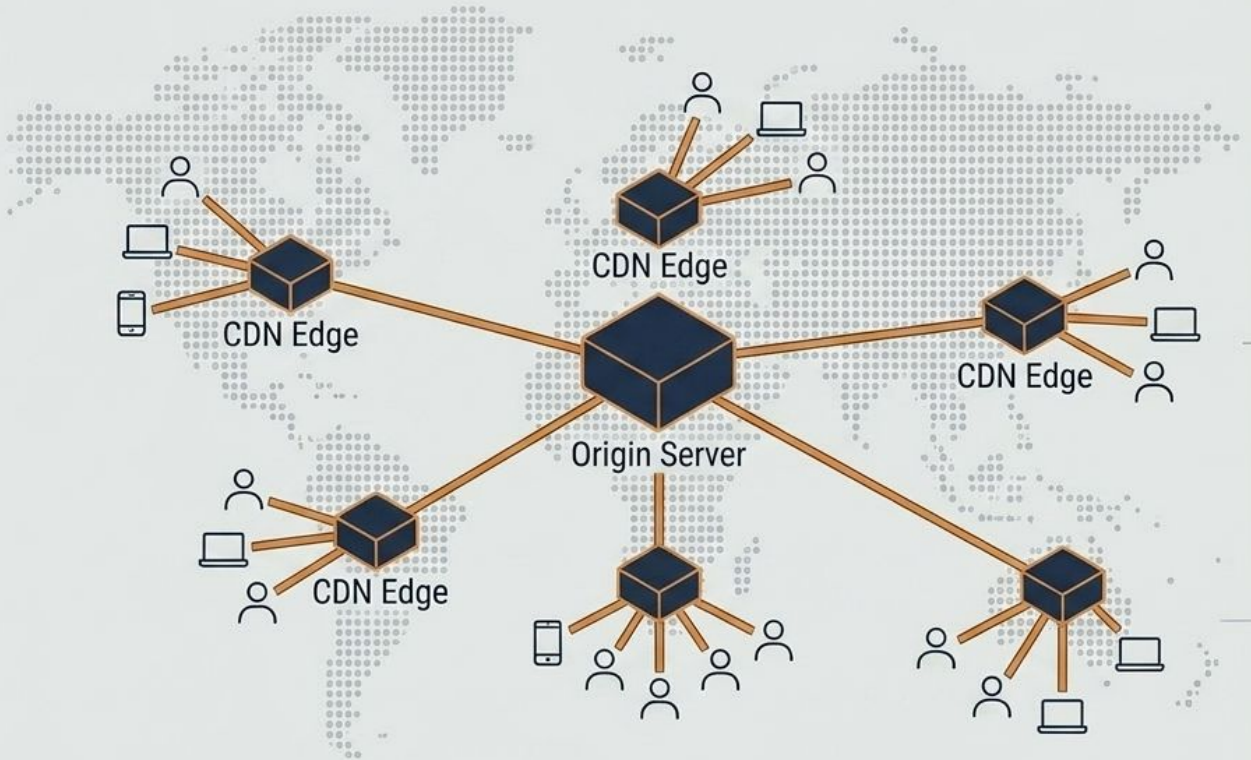
Achieving Global Reach via the Edge

What is a CDN?:
Content Delivery Network. Stores cached content closer to users.

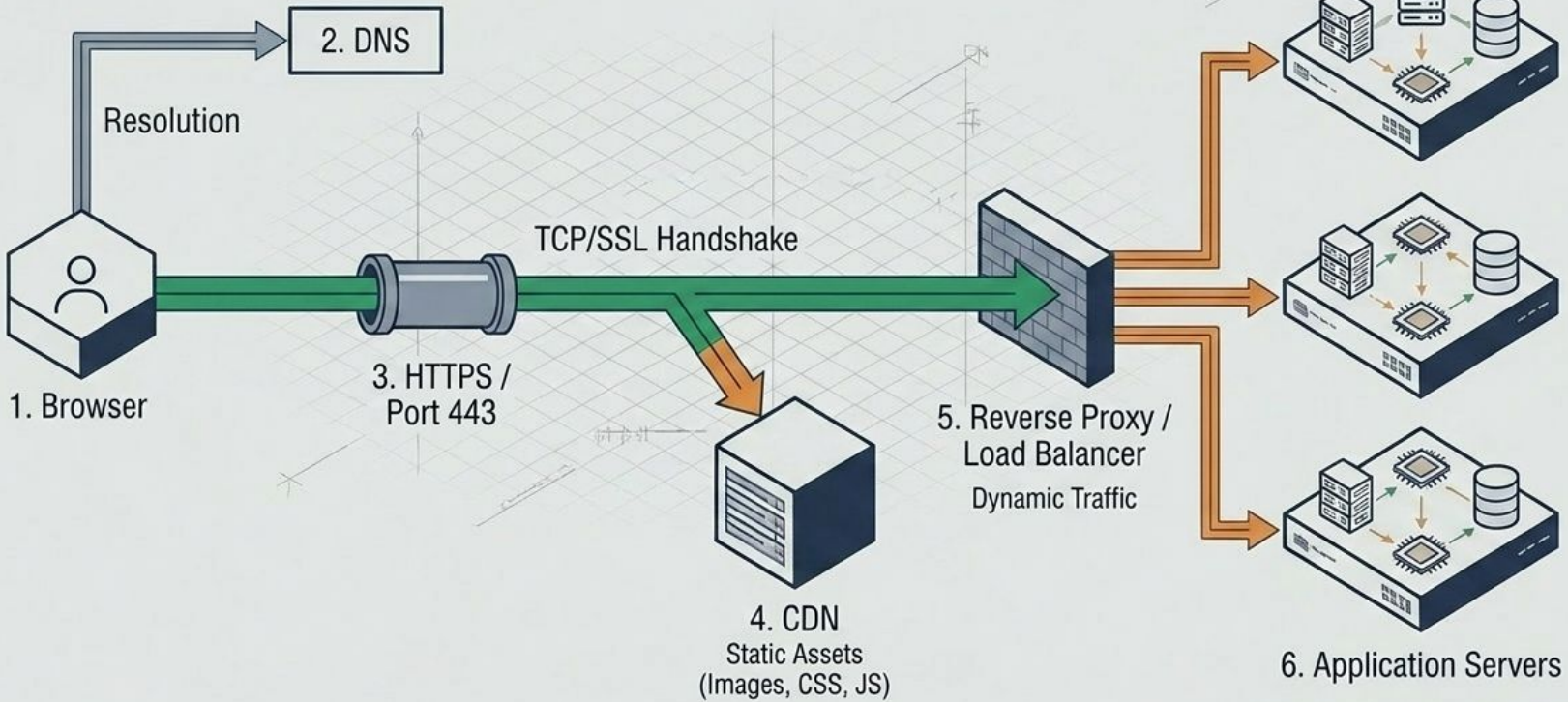
What gets cached?:
Images, Videos, CSS, JavaScript.

System Benefits:
Faster websites, lower latency, reduced server load, DDoS protection, global availability.

Leading Providers:
CloudFront, Cloudflare, Akamai.



Synthesis: Anatomy of a Modern Web Request



Core Competencies Recap

- ✔ OSI vs TCP/IP Models
- ✔ TCP vs UDP Transport
- ✔ DNS Resolution & Records
- ✔ HTTP vs HTTPS Security
- ✔ Common & DevOps Ports
- ✔ Load Balancing & Scaling
- ✔ Reverse Proxies
- ✔ Content Delivery Networks (CDN)

